

MYSHELL

LAB 1. 나의 셸 만들기

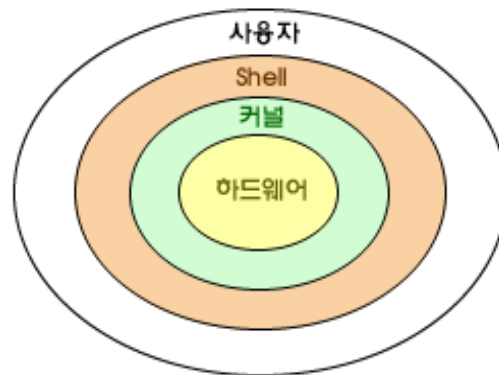


셸이란?

- 사용자가 리눅스를 사용하려면 반드시 로그인해야 한다. 로그인 하게 되면 셸 프로세스가 생성되고 사용자는 항상 셸을 통해 명령을 실행하고 결과를 얻게 된다. 셸을 프로젝트로 진행하기에 앞서 셸에 대해서 살펴보자.

1) 셸의 개념

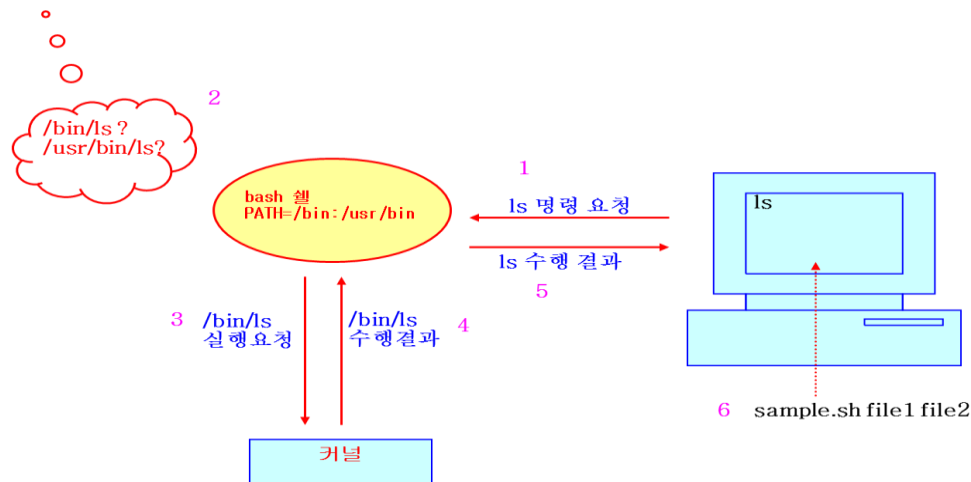
셸(shell)은 하나의 프로그램으로서, 사용자가 리눅스 시스템에 로그인하게되면 커널이 사용자에게 제공하는 무한 루프 (infinite loop) 프로그램이다.



셸이란??

2) 셸의 역할

- 셸은 운영체제의 커널과 사용자 사이의 인터페이스 역할을한다. 사용자가 터미널에서 입력한 명령어는 셸에 의해 타당성 여부를 검증한 후, 타당할 경우엔 자식 프로세스(child process)를 생성해서 해당되는 명령어를 수행시키고, 그렇지 않을 경우에는 해당되는 에러 메시지(error message)를 출력하는 명령어 번역기(Command Interpreter) 프로그램이다.
- 다음은 셸의 명령어 번역 과정을 그림으로 나타낸 것이다.



셸의 기능

- 셸에는 명령을 번역하는 기본 기능 이외에도 다음과 같은 편의 기능이 제공되므로 좀더 편리하게 명령어 사용 환경을 만들 수 있다.

- **alias의 기능**

alias 기능은 사용자의 취향에 맞게 명령어를 재정의하거나, 복잡한 명령어를 간단한 명령어로 정의해 사용할 수 있는 기능이다.

〈사용법〉

alias 사용할명령어= '기존 명령어'

〈사용예〉

alias rm= 'rm -i'

- **history의 기능**

셸에는 먼저 실행했던 명령을 기록하는 기능이 있어 나중에 그 동일한 명령을 수행해야 하는 경우 다시 입력하는 번거로움을 피할 수 있다. history 명령의 수행 결과에서 수행할 명령의 작업 번호나 명령을 확인하고 ! 기호와 함께 입력하면 그 명령을 다시 수행할 수 있다.

〈사용법〉

history

!작업번호

!작업명령

셸의 기능

▪ job control 기능

- job control 기능은 여러 프로그램을 동시에 실행시키면서 각각의 작업 상황을 감시하고 제어하는 기능을 말한다. 작업에는 포그라운드 작업과 백그라운드 작업이 있는데, 포그라운드
- 작업이란 작업 시 주가 되어 실행되는 작업이고, 백그라운드 작업이란 하나의 작업이 처리되고 있는 뒤에서 처리되는 작업이다. 포그라운드로 작업시키면 그 작업이 끝날 때까지 다른 작업을 제어할 수 없는데 비하여 백그라운드 로 작업 시에는 처리 중간에도 작업을 제어할 수 있다.

〈사용법〉

백그라운드 작업 : 명령 &
포그라운드 작업 : 명령

〈사용예〉

sleep 60 &
sleep 5

셸의 기능

■ 명령어 편집 기능

- 히스토리 기능을 통해 기존의 명령을 그대로 재 수행하는 것뿐만 아니라 기존의 명령어를 일부 수정하여 실행할 수도 있는데 이를 명령어 편집 기능 이라고 합니다. 방향키와 BS, DEL 키를 이용하여 자유롭게 명령어 편집을 할 수 있어 명령어 사용을 좀더 편하게 할 수 있다

■ 자동 완성 기능 .

- 명령어 라인에서 파일 이름을 입력할 때 그 파일 이름이 매우 길다면 파일명을 한 글자, 한 글자 모두 입력하는 것이 부담스러울 때가 있다. 리눅스에서 제공하는 셸에는 파일명의 일부분을 입력하여도 긴 파일명 또는 긴 명령을 자동으로 완성시키는 기능이 있다. 단, 같은 이름으로 시작되는 파일 또는 명령이 여러 개 있어서 그 중 어느 것으로 파일명이나 명령으로 완성할 것인지 셸이 정확히 결정할 수 없을 때에는 자신이 완성할 수 있는 곳까지만 완성시키고 나머지는 사용자의 입력을 기다린다.
- 여러분들이 그 파일을 구분할 수 있는 문자를 입력한 후에 <Tab> 키를 입력하면 셸에서 나머지 이름을 완성시켜 준다.

부모 프로세스와 자식 프로세스

- 일반적으로 리눅스에서 프로세스는 부모 프로세스에 의해 생성이 된다. 이렇게 만들어진 프로세스를 자식 프로세스라고 한다. 리눅스에 있는 모든 프로세스는 부모와 자식의 관계를 이루면서 수행된다.
- 프로세스에 관련된 상세 정보에는 프로세스 고유의 식별자인 PID와 부모 프로세스의 PID, 우선순위, CPU 소모 시간, 프로세스 상태 등이 있으며 이 내용은 다음과 같이 'ps -l' 명령을 통해 확인할 수 있다.

```
user@ubuntu: ~/online_lsp/ch04
user@ubuntu:~/online_lsp/ch04$ ps -l
F S  UID  PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S  1000  5139  5130  0  80   0 -  6766 wait  pts/0      00:00:00 bash
0 R  1000  35587  5139  0  80   0 -  3553 -    pts/0      00:00:00 ps
user@ubuntu:~/online_lsp/ch04$
```

- 이와 같이 자식 프로세스를 생성하여 프로그램을 수행하는 방법은 `fork()`, `wait()`, `exec()` 등의 함수를 이용하여 구현된다.

나의 셸 만들기 프로젝트

- 각종 시스템 콜을 이해하는 것도 중요하지만 이 시스템 콜을 언제 어디에 적용해야 하는가도 매우 중요한 요소다. 학습 과정에서 소개되는 단편적인 예제만으로는 적용 능력을 키울 수 없으므로 미니 프로젝트를 통해 학습자들의 적용 능력을 배양하고자 한다.
- 리눅스의 셸은 각종 시스템 콜을 이용하여 구현하기에 적합한 프로그램 구조를 갖추고 있다. 따라서 본 학습 과정에서는 관련 시스템 콜을 학습한 후 나만의 셸을 완성해가는 프로젝트를 기획하였다.
- 프로젝트를 통해 구현하고자 하는 셸의 기능은 다음과 같다.
 - 명령어 실행 기능
 - 빌트인 명령 기능
 - 리다이렉트 기능
 - 파이프 기능
- 각 기능은 관련 시스템 콜 학습 후 LAB 형식으로 구현해 나간다.

셸의 기본 구조 구현

- 셸은 사용자의 입력을 기다리는 무한 루프 구조의 프로그램이다. exit 명령이 입력되면 셸을 종료하도록 되어 있다. 그리고 셸에서 명령을 입력 받으면 공백 문자를 구분자로 명령어와 옵션 및 인수들을 분리하고 환경 변수 PATH 에 설정되어있는 경로의 실행 파일을 찾아 해당 명령을 실행한다.
- 이 부분을 차례대로 프로그램으로 완성해보자
- 단계 1. 사용자의 입력을 기다리는 무한 루프의 구조 프로그램으로 exit 명령 입력 시 셸을 종료하도록 구현해보자 (main.c)

셸의 기본 구조 구현

- 단계 2. 셸에서 명령을 입력 받으면 명령어 부분과 각 옵션 및 인자들을 공백 문자를 기준으로 분리해야 한다. 즉 구문 분석 과정이 필요하다. 이에 관련된 함수를 작성해보자.
(parsing.c)

`command_parse( )`

인수 : 입력 받은 줄

반환 : 분석된 일련의 문자열

`comman_freelist( )`

인수 : 일련의 문자열

반환 : 없음



main.c 에서 `command_parse( )`와 `comman_freelist( )`가 호출되도록 수정해보자.

셸의 기본 구조 구현

- 단계2에서 완성한 소스 프로그램을 컴파일하여 실행해보자. 여기서 소스코드가 앞으로도 추가될 것을 고려하여 다음과 같이 Makefile 을 작성하여 컴파일하자

Makefile	
1	<code>OBJS = main.o parsing.o</code>
2	<code>SRCS = \$(OBJS:.o=.c)</code>
3	
4	<code>minishell: \$(OBJS)</code>
5	<code>\$(CC) -o \$@ \$(OBJS)</code>
6	<code>.c.o: \$(SRCS)</code>
7	<code>\$(CC) -c \$<</code>
8	
9	<code>clean:</code>
10	<code>rm \$(OBJS) minishell</code>

명령어 줄은 공백이 아니라 TAB 문자로 시작해야함!!

셸의 기본 구조 구현

- 단계 3. 이제 명령어에 대한 구문 분석이 가능해 졌으니 minishell에서 리눅스의 각종 명령을 처리할 수 있도록 프로그램을 작성해보자. 명령은 다음과 같이 크게 두 종류로 나눌 수 있다.
 - 빌트인 명령 : 셸 내에서 함수방식으로 처리
 - 외부 명령 : 외부의 실행 파일 수행
- 우선 빌트인 명령을 위해 다음과 같은 구조체와 관련 함수를 minishell.h 헤더파일에 선언하고, 호스크 명을 출력하는 빌트인 명령 hostname 을 처리해보자 .

헤더파일(minishell.h)

```
#ifndef MINISHELL_H_
#define MINISHELL_H_

char** command_parse(char*line);
void command_freelist(char**arglist);

void hostname_func(char **);
int check_builtin(char **);

typedef struct bt_type {
    char *cmd;
    void (*bt_func)(char **);
} BT_TYPE;

#endif /* MINISHELL_H_ */
```

소스코드 (main.c)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "minishell.h"

#define MAXLINE 255

int main(void) {
    char line[MAXLINE];
    int i;
    char** arglist;

    fputs("minishell> ", stdout);

    while (fgets(line, MAXLINE, stdin)) {
        if (!strncmp(line, "exit", 4))
            exit(0);
        if (arglist = command_parse(line)) {
            if (!check_builtin(arglist)) {
                for (i = 0; arglist[i]; i++) {
                    printf("[%d] : %s\n", i, arglist[i]);
                }
            }
            command_freelist(arglist);
        }
        fputs("minishell> ", stdout);
    }
    return 0;
}
```

소스코드 (isbuiltin.c)